🚂 Indian Railways Dynamic ETA API (v1.5)

Smart India Hackathon (SIH) Prototype - Operational Update

This API utilizes an Optuna-optimized Stacked Meta-Model (LightGBM + XGBoost + CatBoost) to process static scheduling data, weather, and **real-time operational metrics** (live GPS speed, current delay carryover, downstream congestion, and unscheduled stops) to predict dynamic ETAs.

📍 Base URL

- **Local Development:** `http://127.0.0.1:8000`
- **Public/Ngrok URL:** `https://acquire-wired-alike.ngrok-free.dev`
- **Swagger UI (Interactive Docs):** `/docs`

🚀 Endpoint: Predict ETA

- **Path:** `/predict_eta`
- **Method:** `POST`
- **Content-Type:** `application/json`

📩 Request Body (JSON)

The API requires the following fields. If a live operational metric (like speed, current delay, or trains ahead) drops out, the API will automatically use safe fallbacks to prevent crashes.

Field Name	Type	Example	Description / Notes
`train_number`	Integer	`12004`	Train identifier
`departure_hour`	Integer	`14`	Hour of departure (0-23)
`month`	Integer	`12`	Month of journey (1-12)
`is_night_departure`	Integer	`0`	1 = Yes, 0 = No
`is_peak_hour`	Integer	`1`	1 = Yes, 0 = No
`is_monsoon_season`	Integer	`0`	1 = Yes, 0 = No
`is_fog_risk`	Integer	`1`	1 = Yes, 0 = No
`fog_risk_score`	Float	`0.85`	0.0 (Clear) to 1.0 (Dense Fog)
`season_severity_score`	Float	`0.6`	0.0 (Mild) to 1.0 (Severe)
`track_doubled`	Integer	`1`	1 = Double track, 0 = Single
`is_electrified`	Integer	`1`	1 = Electric, 0 = Diesel
`is_hdn_route`	Integer	`1`	1 = High Density Network

Field Name	Type	Example	Description / Notes
`distance_km`	Float	`145.5`	Total distance of journey
`distance_completed_km`	Float	`85.5`	**[LIVE]** Distance already covered
`num_scheduled_stops`	Integer	`3`	Number of scheduled halts
`scheduled_travel_hours`	Float	`2.5`	Expected travel time (hours)
`psr_count`	Integer	`4`	Permanent Speed Restrictions count
`loco_age_years`	Integer	`12`	Locomotive age
`coach_age_years`	Integer	`8`	Coach age
`maintenance_score`	Integer	`7`	1-10 Scale
`seat_utilisation_pct`	Integer	`110`	Passenger load (%)
`zone_congestion_index`	Float	`0.9`	0.0 (Empty) to 1.0 (Gridlock)
`route_historical_ontime_pct`	Integer	`65`	Historical punctuality (0-100)
`late_incoming_rake`	Integer	`1`	1 = Previous journey delayed
`source_station_category`	String	`"A1"`	Origin class (A1, A, B, C, D, E)
`destination_station_category`	String	`"B"`	Target class
`live_speed_kmh`	Float	`42.5`	**[LIVE]** GPS Ping (Defaults to 0.0)
`current_delay_minutes`	Float	`25.0`	**[LIVE]** Delay accumulated so far
`trains_ahead`	Integer	`2`	**[LIVE]** Downstream block congestion
`unscheduled_stop_count`	Integer	`1`	**[LIVE]** Unplanned halts so far
`primary_delay_cause`	String	`"Normal Running"`	**[LIVE]** Incident Log
`scheduled_arrival_time`	String	`"2026-10-15 16:30:00"`	`YYYY-MM-DD HH:MM:SS` format

📩 Response Body (JSON)

The API returns the calculated delay, the final dynamic ETA (formatted cleanly), and a generated reason for the delay.


```
{
  "train_number": 12004,
  "destination_station": "B",
  "status": "Running Late",
  "primary_delay_factor": "Heavy Fog/Visibility Restrictions",
  "live_speed_kmh": 42.5,
  "current_delay_minutes": 25.0,
  "trains_ahead": 2,
  "unscheduled_stop_count": 1,
  "scheduled_arrival": "2026-10-15 16:30:00",
  "predicted_delay_minutes": 147.6,
  "dynamic_eta": "2026-10-15 18:57:36"
}
```


💻 Developer Code Snippets

For Frontend (JavaScript / React)

```
const getDynamicETA = async (trainData) => {
  const response = await fetch("YOUR_NGROK_URL/predict_eta", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(trainData)
  });

  const result = await response.json();
  console.log("Predicted ETA:", result.dynamic_eta);
  console.log("Delay Factor:", result.primary_delay_factor);
  return result;
};
```

For Backend / Microservices (Python)

```
import requests

url = "YOUR_NGROK_URL/predict_eta"
payload = {
    # ... insert data dictionary from above table
}

response = requests.post(url, json=payload)
data = response.json()
print(f"Train {data['train_number']} ETA: {data['dynamic_eta']}")
```